

# Guess Market

## Exercise 2 - JavaFX User Manual and Implementation Notes

---

| Participant | ID        | Email                 |
|-------------|-----------|-----------------------|
| Amir Dayyef | 323092023 | amirdayyef@gmail.com  |
| Nour Guty   | 322235516 | nourguty581@gmail.com |

**Bonuses: No Exercise 2 bonus features are claimed. The submission focuses on the complete required functionality.**

Source repository: [github.com/AmirDayyef/guess-market-ex1-amir-dayyef](https://github.com/AmirDayyef/guess-market-ex1-amir-dayyef)

# 1. Quick start

1

Extract the complete archive

Keep the main JAR, run.bat, and the lib directory together. Do not run the application from inside the ZIP file.

2

Verify Java

The computer must have JDK 25 available through the PATH environment variable. The supplied JavaFX and JAXB runtime JARs are already included.

3

Start the program

Double-click run.bat, or execute it from Command Prompt. The script changes to its own directory, so it can be launched from any working directory.

4

Load market data

Click Load XML and choose a valid Exercise 2 XML file from any local directory. The file chooser supports paths containing spaces.

5

Choose the acting user

Select a user in the Users tab or use the Acting as selector at the top. Every lifecycle and trading action is performed as that user.

## XML loading behavior

Loading runs in a JavaFX Task, displays an indeterminate progress indicator, and includes the required short simulated delay. A valid file completely replaces the previous market. An invalid file displays a detailed error and leaves the previous valid market untouched.

- Every user name must be unique and every initial balance must be greater than zero.
- Every market-maker event reference must point to an existing event.
- Every event must have exactly one market maker.
- Each event must have exactly two distinct options and exactly one supported trading method.

# 2. User interface guide

## Events tab

The Events tab lists every inactive, active, and closed event. Filters can be combined by status, trading method, and commission method. Select an event to inspect its contract balance, options, pricing data, open orders, participants, trade history, and settlement result.

- Only the assigned market maker can open an inactive event or close an active event.
- LMSR events show current option prices and direct share-purchase controls.
- Order Book events show a separate book for each option plus LAST, BID, ASK, MID, and SPREAD values.

## Users tab

Selecting a user makes that person the acting user. The screen shows the current balance, blocked state, owned events, participating events, holdings, commissions, and profit or loss. A separate active-markets list lets a user make their first trade even when they do not yet have a position.

Choose a related or active event to open the complete event details and trading panel inside the Users tab. This mirrors the event-side information while keeping every action tied to the selected user.

### 3. Trading and lifecycle rules

#### Opening an event

- Events are inactive immediately after XML loading and cannot be traded until opened.
- For LMSR, the market maker pays the initial subsidy  $C(0,0)$  into the event account.
- For Order Book, the market maker pays the configured initial amount and receives  $\text{floor}(\text{initial} / d)$  pairs of complementary shares.
- Opening is rejected before any state changes if the market maker lacks the required balance or the event is not inactive.

#### LMSR purchases

Choose an option and a positive whole-number quantity. The engine calculates the exact LMSR cost difference, applies an on-purchase commission when configured, credits that commission to the market maker, updates the event collateral, and records the trade.

#### Order Book orders

- Choose an option, BUY or SELL, a positive quantity, and a price greater than zero and no higher than  $d - 0.01$ .
- An incoming order walks compatible resting orders using best-price then oldest-order priority. The resting order price is the execution price.
- One incoming order may fill several resting orders. Any unfilled remainder stays in the book.
- A SELL order is rejected when the user does not own enough shares after subtracting shares already committed to other open SELL orders.
- When peer mint is enabled, complementary BUY orders whose prices sum to at least  $d$  mint the minimum remaining quantity. The resting buyer pays the resting quote; the incoming buyer pays the complement to  $d$ .

#### Commissions and negative balances

On-purchase commission is paid by each buyer and credited directly to the event market maker. On-close commission is deducted from winning payouts and credited to the market maker. A submitted action is allowed to complete even if it makes the user's balance negative; the user is then warned and blocked from all future actions.

#### Closing and settlement

Only the market maker can choose the winning option and close an active event. Winning holdings receive 1.00 per LMSR share or  $d$  per Order Book share. Open orders are cancelled, closing commission is transferred, remaining collateral returns to the market maker, and the event account becomes zero.

### 4. Important implementation decisions

- If the Order Book initial investment is not divisible by  $d$ , the unpaired remainder stays as event collateral and returns to the market maker during settlement.

- All monetary calculations use double precision internally and are displayed with two decimal places in the UI.
- Option selection is numeric inside the engine and is presented as a controlled choice in the UI, preventing case or spelling errors.
- The application window remains resizable. Split panes, wrapping action rows, compact tables, and scroll panes preserve access to all information at smaller sizes.

## 5. Architecture and main classes

|                           |                                                                                                                                                |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| GuessMarketEngine         | Public UI-independent contract. Exposes loading, queries, lifecycle operations, LMSR purchases, Order Book orders, and persistence operations. |
| GuessMarketEngineImpl     | Coordinates validated state changes, users, market makers, money flow, order matching, minting, settlement, and immutable DTO creation.        |
| MarketState               | Owens the currently loaded events, users, and normalized source-file path.                                                                     |
| MarketEvent               | Stores one event's configuration and lifecycle state, contract balance, options, open orders, trades, and settlement totals.                   |
| MarketUser / UserPosition | Track account balance, blocked state, MM ownership, per-event holdings, paid amounts, commissions, and profit or loss.                         |
| MarketOrder               | Represents one price-time-prioritized BUY or SELL instruction and its remaining quantity.                                                      |
| MarketOption / Trade      | Store share totals, pricing statistics, and immutable execution records.                                                                       |
| XmlMarketLoader           | Securely unmarshals and validates a complete candidate state before replacing the live market.                                                 |
| XmlMarketFile             | JAXB-only representation of the Exercise 2 XML structure.                                                                                      |
| GuessMarketApplication    | Builds the JavaFX screens, filters and tables; starts background XML loading and converts UI actions into interface calls.                     |
| Launcher                  | Small non-JavaFX entry point used by the batch file to start the application reliably from the classpath.                                      |

### Module boundary

The JavaFX module depends on the API and engine modules. The engine does not import JavaFX and never knows which UI is calling it. UI state uses ordinary JavaFX controls, while engine data crosses the boundary only through immutable DTO records and the GuessMarketEngine interface.

## 6. Verification performed

|                      |                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------|
| Maven reactor        | API, engine, and JavaFX modules compile successfully with JDK 25.                                         |
| Automated suite      | 10 tests pass with zero failures and zero errors.                                                         |
| small.xml            | Accepted: 2 events and 3 users.                                                                           |
| multiple.xml         | Accepted: 4 events and 3 users.                                                                           |
| error-2.xml          | Rejected with the specific reason that user Avrum has an initial balance that is not greater than zero.   |
| error-3.xml          | Rejected with the specific reason that user Avrum references nonexistent event 12 as market maker.        |
| Failure preservation | After both invalid loads, the previously loaded 4-event market remained available.                        |
| Runtime smoke test   | The packaged JavaFX application launches on JDK 25 and displays the resizable Events and Users dashboard. |

For grading, keep the submission directory intact and begin with run.bat. If startup fails, first verify that java -version reports Java 25 and that the lib directory has not been renamed or partially copied.